

CUSTOM LOSS FUNCTION

Prepared By :

Yogesh Kumar(23124052)

*Swayam Bhushan
Singh(23124047)*

Abstract:

Loss functions are fundamental to training machine learning models, as they quantify the discrepancy between predicted and actual outcomes, steering the optimization process. While traditional loss functions like Mean Squared Error (MSE) and Cross-Entropy are widely employed, they often fall short in capturing the unique requirements of certain tasks, particularly in time-series forecasting. These standard methods may struggle to enforce desirable properties such as prediction smoothness, which is critical in applications like weather forecasting or stock price prediction.

To address these limitations, we propose **Custom_Loss**, a custom loss function tailored for time-series data. Custom_Loss combines the predictive accuracy of MSE with an additional smoothness penalty term that discourages abrupt changes between consecutive predictions. By incorporating this smoothness constraint, the loss function ensures that predictions not only minimize the error relative to the true values but also maintain a stable and realistic flow across time steps. This dual-objective approach balances accuracy and consistency, enabling models to produce predictions that are both precise and coherent, making Custom_Loss particularly suited for dynamic and sequential data forecasting tasks.

CONTENTS

1. Introduction

2. Methodology

2.1 Combination Loss Functions

2.2 MSE(Mean Squared Error)

2.3 Smothness Penalty

3. Experiments

3.1 Jena City Climate Dataset

3.1.1 Plotting the Loss of Model 1

3.1.2 Metrics on Model 1

3.1.3 Plotting the loss of Model 2

3.1.4 Metrics on Model 2

3.2 S&P(^GSPC) Stock Price Dataset

3.2.1 Plotting the loss

3.2.2 Test Data Plots

3.2.3 Metrics on Test Data

3.3 Google Finance Dataset

3.3.1 Plotting the Loss

3.3.2 Test Data Plots

3.3.3 Metrics on Test Data

4. Result

1. Introduction :

Custom_Loss builds on the foundational Mean Squared Error (MSE) loss widely used in machine learning models, enhancing it with a specialized penalty term tailored for time-series forecasting. This loss function combines the predictive accuracy of MSE with a smoothness penalty that encourages consistency and harmony in the predicted time-series flow. The penalty is applied when consecutive predictions exhibit abrupt changes, with the magnitude of the penalty increasing based on the severity of these changes.

By integrating this smoothness constraint alongside the traditional MSE component, **Custom_Loss** enables models to achieve not only lower overall prediction error but also smoother and more coherent predictions over time. This dual-objective design is particularly valuable in time-series tasks, such as weather forecasting and stock price prediction, where maintaining the continuity of trends is as crucial as ensuring accurate values.

2. Methodology :

2.1 Combination of loss functions :

The custom loss function you've implemented for **Custom_Loss** combines two components:

- **Mean Squared Error (MSE)** loss between the true and predicted values, ensuring the model minimizes the overall prediction error.
- **Smoothness Penalty**, which discourages abrupt changes between consecutive predictions, encouraging a stable and continuous flow in the predicted time series. This penalty helps maintain consistency over time by penalizing large fluctuations in predictions, ensuring smoother transitions between time steps.

2.2 MeanSquaredError(MSE):

The MSE loss is computed between the true values

$$\text{mse_loss} = \frac{1}{n} \sum_{i=1}^n (y_{\text{true}} - y_{\text{pred}})^2$$

2.3 Smoothness Penalty :

The **Smoothness Penalty** measures the absolute difference between each consecutive pair of predictions in the time series. It sums the absolute differences between the predicted value at time step i and the predicted value at the next time step $i+1$, then averages them over all consecutive pairs.

The penalty ensures that if there are sudden jumps or sharp changes between consecutive predictions, the model is penalized. This encourages the model to generate predictions that change gradually over time, maintaining smooth transitions in the time series, rather than producing erratic or volatile predictions.

$$\text{SmoothnessPenalty}(y_{\text{pred}}) = \frac{1}{N-1} \sum_{i=1}^{N-1} |y_{\text{pred},i+1} - y_{\text{pred},i}|$$

Custom_Loss = lambda_1(MSE) +lambda_2(Smoothness Penalty)

$$L(y_{\text{true}}, y_{\text{pred}}) = \lambda_1 \cdot \text{MSE}(y_{\text{true}}, y_{\text{pred}}) + \lambda_2 \cdot \text{SmoothnessPenalty}(y_{\text{pred}})$$

Where λ_1 and λ_2 are hyperparameters that control the relative importance of the MSE and smoothness penalty terms.

3. Experiments :

- 3.1 Jena city climate dataset : The dataset being used for this experiment is the **Jena Climate Dataset**, which contains multiple time-based meteorological values recorded since 2009. For this analysis, we focus exclusively on the **temperature (T (degC))** readings to perform time-series forecasting.

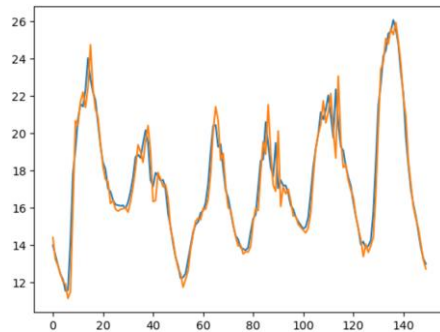
The data is preprocessed, scaled, and passed to multiple models to evaluate the performance of the custom loss function compared to conventional ones. Each model is trained for 50 epochs with identical training parameters, and the graph of the loss function is plotted to analyze the results.

```
df.describe()
```

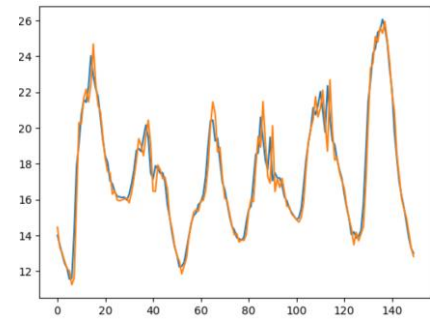
	p (mbar)	T (degC)	Tpot (K)	Tdew (degC)	rh (%)	VPmax (mbar)	VPact (mbar)	VPdef (mbar)	
count	420551.000000	420551.000000	420551.000000	420551.000000	420551.000000	420551.000000	420551.000000	420551.000000	420551.000000
mean	989.212776	9.450147	283.492743	4.955854	76.008259	13.576251	9.533756	4.042412	
std	8.358481	8.423365	8.504471	6.730674	16.476175	7.739020	4.184164	4.896851	
min	913.600000	-23.010000	250.600000	-25.010000	12.950000	0.950000	0.790000	0.000000	
25%	984.200000	3.360000	277.430000	0.240000	65.210000	7.780000	6.210000	0.870000	
50%	989.580000	9.420000	283.470000	5.220000	79.300000	11.820000	8.860000	2.190000	
75%	994.720000	15.470000	289.530000	10.070000	89.400000	17.600000	12.350000	5.300000	
max	1015.350000	37.280000	311.340000	23.110000	100.000000	63.770000	28.320000	46.010000	

3.1.1 Plotting the Graphs of model 1:

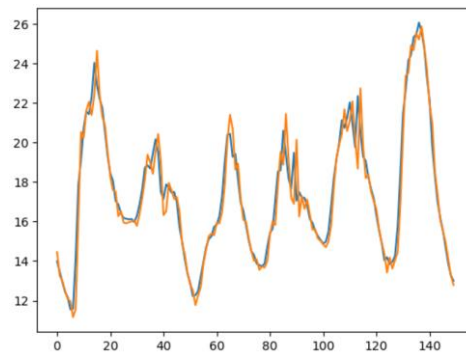
The following plots illustrate how the model 1 fit the data using different loss functions, providing a comparative analysis of their performance.



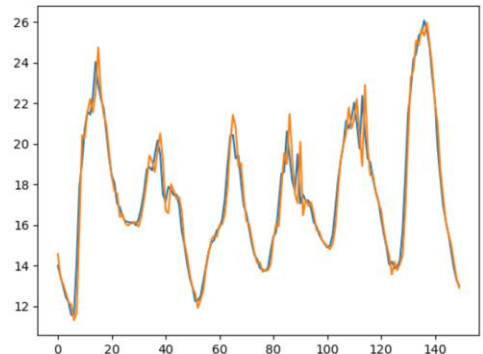
Mean Absolute Loss



Mean Squared Loss



Huber Loss



Custom Loss(Custom_Loss)

* True values are plotted by BLUE and ORANGE plots predicted values

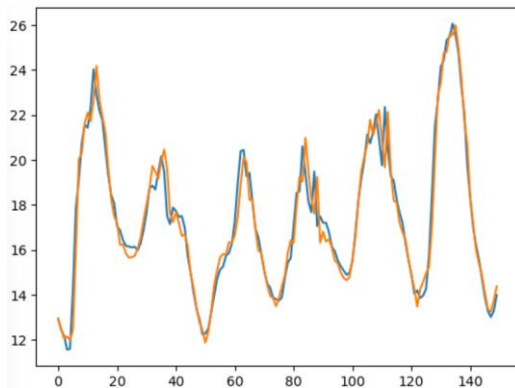
* We are only plotting some parts of test data not the whole test data

3.1.2 Metrics on Test Data of Model 1:

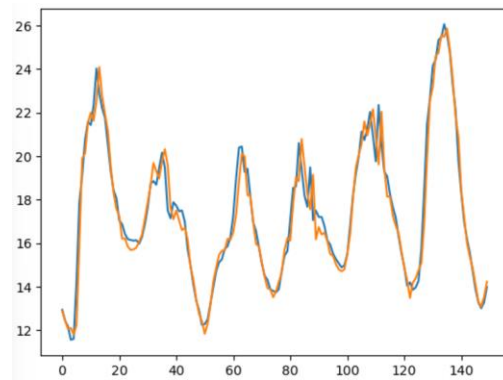
Loss Used	MSE	MAE	RMSE	R2 score
Mean squared loss	0.58654572	0.52120954	0.76586273	0.99157506
Mean absolute loss	0.60265289	0.52511751	0.77630721	0.99134371
Huber loss	0.59114347	0.52295504	0.76885855	0.99157506
Custom_Loss	0.58482493	0.52196408	0.76473847	0.99159978

3.1.3 Plotting the Graphs of Model 2:

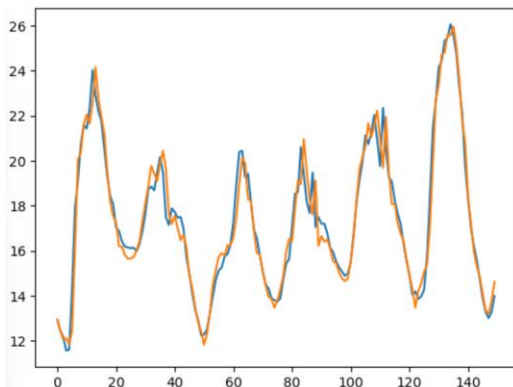
The following plots illustrate how model 2 fit the data using different loss functions, providing a comparative analysis of their performance.



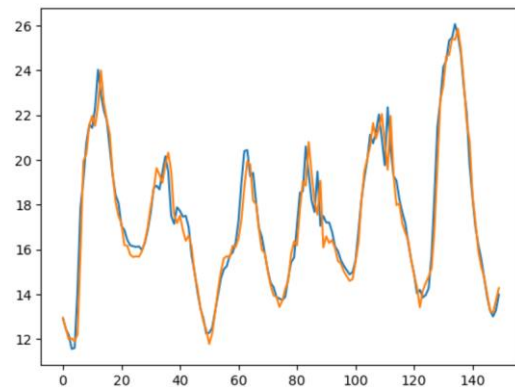
Mean Squared Loss



Mean Absolute Loss



Huber Loss



Custom Loss(Custom_Loss)

3.1.4 Metrics on Test Data of Model 2:

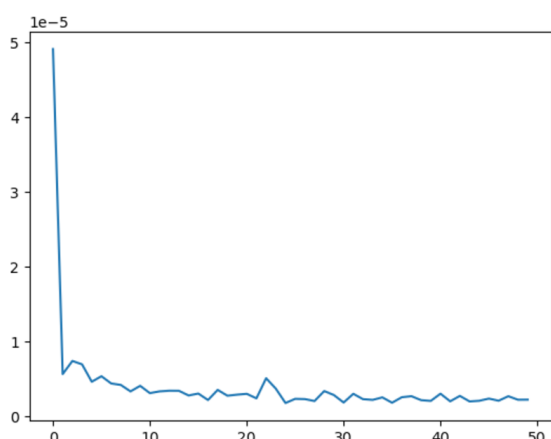
Loss Used	MSE	MAE	RMSE	R2 score
Mean squared loss	0.44679710	0.46134354	0.66842883	0.993584
Mean absolute loss	0.45060931	0.45575873	0.67127439	0.993527
Huber loss	0.44089567	0.45818937	0.66399975	0.993667
Custom_Loss	0.43972482	0.45856769	0.66311750	0.993686

Our team has achieved improved efficiency with our custom loss function. The custom loss function demonstrates a higher R2 score compared to the other three loss functions.

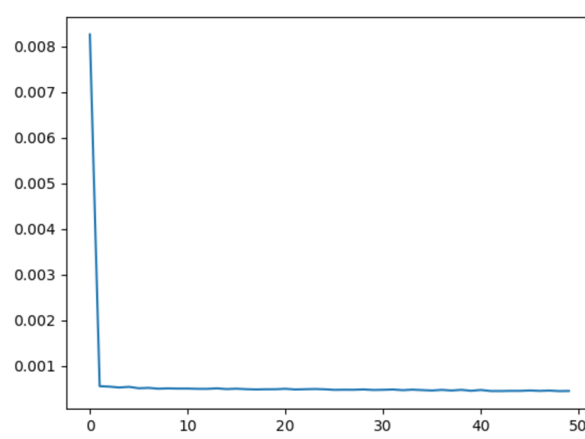
- 3.2 S&P(^GSPC) Stock Price Dataset : The dataset used for the first test case is the **^GSPC dataset**, which contains historical financial data from 1930 to 2024-01-01. For this analysis, we focus exclusively on the **Adjusted Close (Adj Close)** value for time-series forecasting. The data is scaled and passed to multiple models to evaluate the performance of the custom loss function in comparison to conventional ones. Each model is trained for 50 epochs with identical training parameters, and the loss function's behavior is analyzed through plotted graphs.

Price	Adj Close	Close	High	Low	Open	Volume
Ticker	^GSPC	^GSPC	^GSPC	^GSPC	^GSPC	^GSPC
count	24114.000000	24114.000000	24114.000000	24114.000000	24114.000000	2.411400e+04
mean	608.571427	608.571427	612.053256	604.618513	588.512004	8.879113e+08
std	974.099095	974.099095	979.447670	968.029907	985.101725	1.604973e+09
min	4.400000	4.400000	4.400000	4.400000	0.000000	0.000000e+00
25%	24.559999	24.559999	24.559999	24.559999	9.620000	1.490000e+06
50%	101.814999	101.814999	102.540001	101.000000	41.725000	1.973000e+07
75%	993.612518	993.612518	1002.412491	982.717484	992.974991	8.969750e+08
max	4796.560059	4796.560059	4818.620117	4780.979980	4804.509766	1.145623e+10

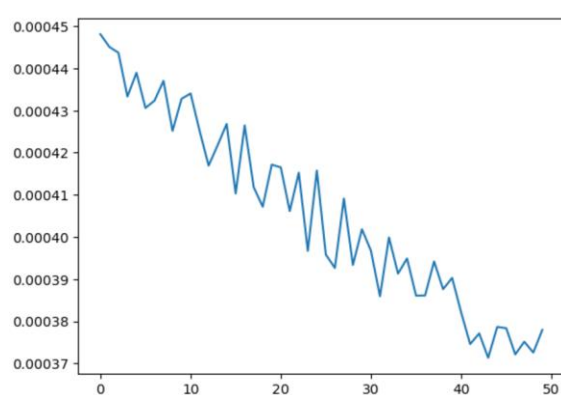
3.2.1 Plotting the Loss :



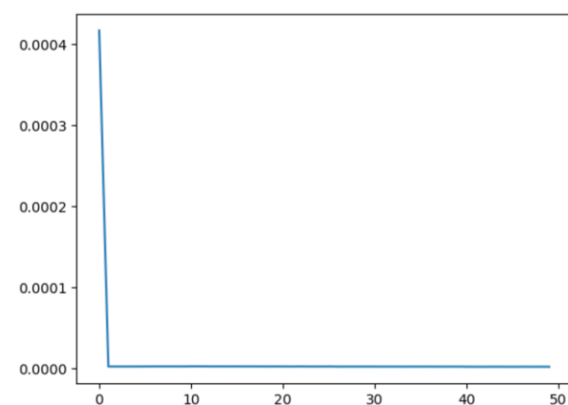
Mean Squared loss



Mean Absolute Loss

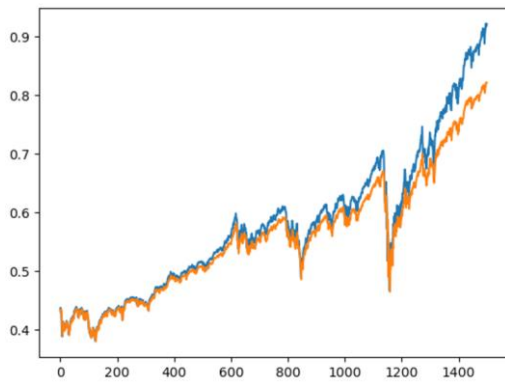


Huber Loss

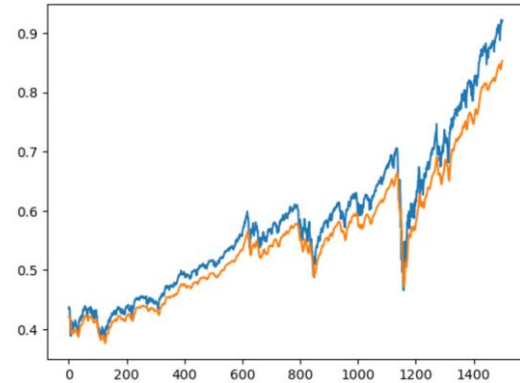


Custom Loss(Custom_Loss)

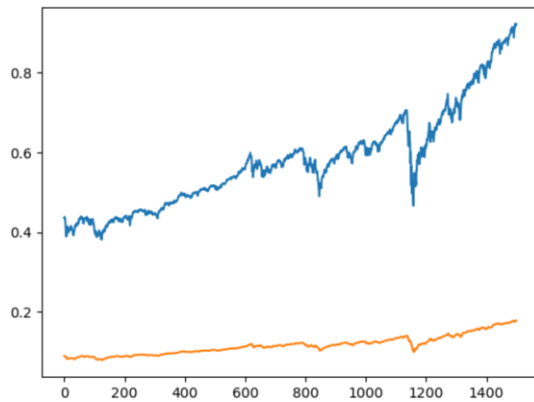
3.2.2 Test Data Plots :



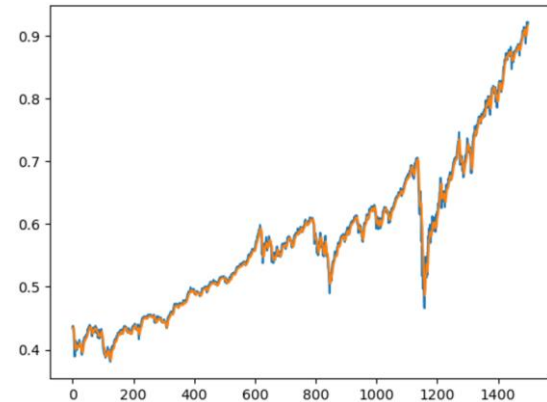
Mean Square Loss



Mean Absolute Loss



Huber Loss



Custom_Loss

3.2.3 Metrics on Test Data :

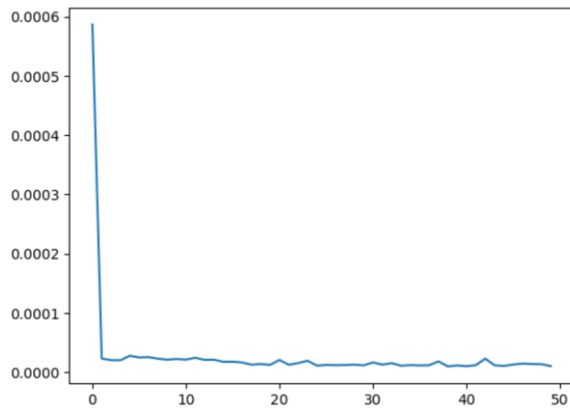
Loss Used	MSE	MAE	RMSE	R2 score
Mean squared loss	0.00297031	0.03953032	0.05450055	0.90646292
Mean absolute loss	0.00191602	0.03825838	0.04377237	0.93966321
Huber loss	0.30732715	0.53497402	0.55437095	Very Low
Custom_Loss	0.00010224	0.00723330	0.01011186	0.99678008

Our team has achieved improved efficiency with our custom loss function. The custom loss function demonstrates a higher R2 score compared to the other three loss functions.

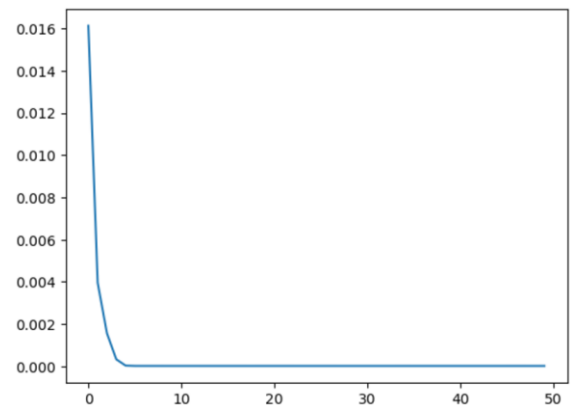
3.3 Google Finance Dataset : The dataset used for the first test case is the **Google finance dataset**, which contains historical financial data from 1995 to 2024-01-01. For this analysis, we focus exclusively on the **Adjusted Close (Adj Close)** value for time-series forecasting. The data is scaled and passed to multiple models to evaluate the performance of the custom loss function in comparison to conventional ones. Each model is trained for 50 epochs with identical training parameters, and the loss function's behavior is analyzed through plotted graphs.

Price	Adj Close	Close	High	Low	Open	Volume
Ticker	GOOGL	GOOGL	GOOGL	GOOGL	GOOGL	GOOGL
count	4875.000000	4875.000000	4875.000000	4875.000000	4875.000000	4.875000e+03
mean	41.813308	41.916845	42.346107	41.468347	41.906119	1.201621e+08
std	38.563467	38.658957	39.074717	38.225827	38.638828	1.492601e+08
min	2.496571	2.502753	2.546046	2.401401	2.482232	9.312000e+06
25%	12.879769	12.911662	13.043293	12.781281	12.924424	3.176400e+07
50%	27.023087	27.090000	27.290501	26.858000	27.104000	6.412781e+07
75%	58.212107	58.356249	58.834251	57.576750	58.012501	1.458980e+08
max	149.468399	149.838501	151.546494	148.899002	151.250000	1.643023e+09

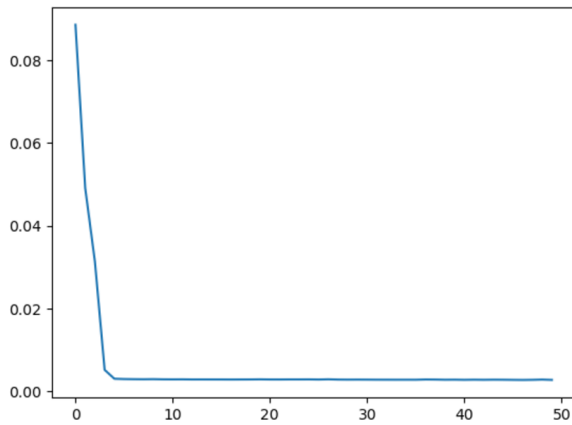
3.3.1 Loss Plots :



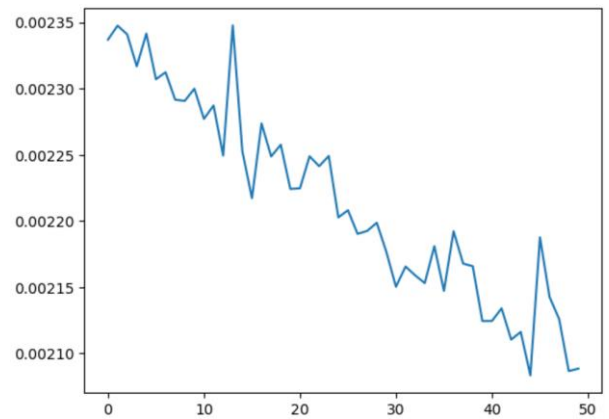
Mean Squared Loss



Custom_Loss

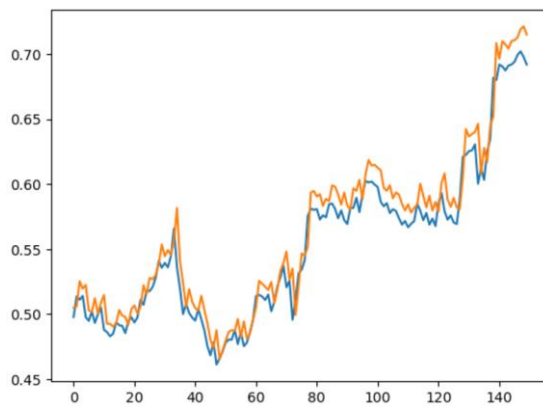


Mean Absolute Loss

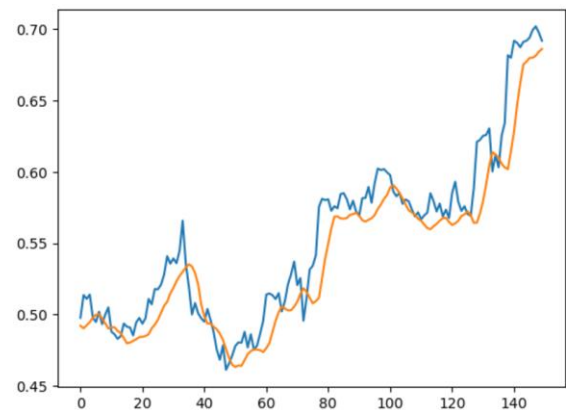


Huber Loss

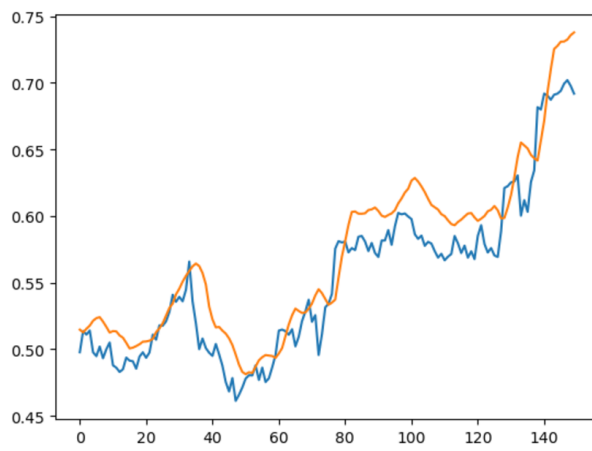
3.3.2 Test Data Plots :



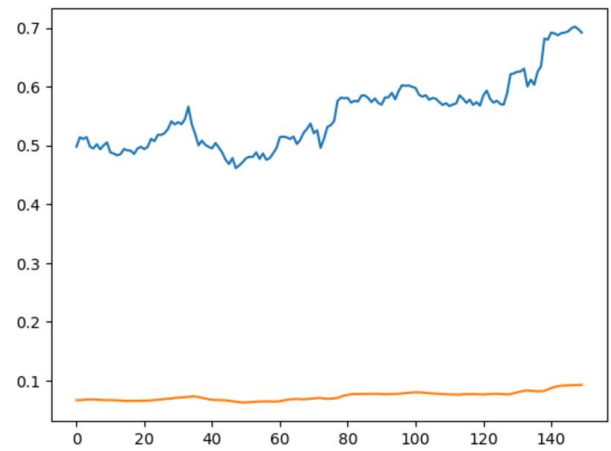
Mean Square Loss



Custom_Loss



Mean Absolute Loss



Huber Loss

Loss Used	MSE	MAE	RMSE	R2 score
Mean squared loss	0.00072456	0.02223557	0.02691782	0.96835264
Mean absolute loss	0.00352042	0.05064927	0.05933314	0.84623686
Huber loss	0.43478882	0.64602214	0.65938519	Very low
Custom_Loss	0.00059425	0.01923587	0.02437743	0.97404424

Our team has achieved improved efficiency with our custom loss function. The custom loss function demonstrates a higher R2R2 score compared to the other three loss functions.

Result :

In conclusion, the custom loss function developed in this project effectively balances prediction accuracy and the smoothness of the output time series. By combining the **Mean Squared Error (MSE)** with a **smoothness penalty**, it ensures that the model minimizes both the error relative to the true values and the abrupt changes between consecutive predictions.

This approach is particularly valuable for time-series forecasting tasks where maintaining a coherent and realistic flow of predictions—such as in temperature forecasting or other sequential data modeling—is crucial. The inclusion of the smoothness penalty allows the model to learn patterns that are not only accurate but also consistent over time, reflecting the natural dynamics of the data.

The dual-objective nature of the loss function facilitates a more refined training process, offering improved performance in scenarios where both precision and temporal consistency are essential. Further testing and refinement can enhance its robustness and adaptability to other domains, making it a powerful tool for tackling complex time-series modeling challenges.

